

main.py



Share

Run

Output

Clear

```
1 text = input("Enter text: ")
2 key = int(input("Enter key: "))
3
4 result = ""
5
6 for ch in text:
7     if ch.isalpha():
8         result += chr((ord(ch.upper()) - 65 + key) % 26 + 65)
9     else:
10        result += ch
11
12 print("Encrypted Text:", result)
```

Enter text: HELLO
Enter key: 3
Encrypted Text: KHOOR

=== Code Execution Successful ===



```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n,i,j,temp;
6     int bt[10],p[10],wt[10],tat[10];
7
8     printf("Enter number of processes: ");
9     scanf("%d",&n);
10
11     for(i=0;i<n;i++)
12     {
13         p[i]=i+1;
14         printf("Burst time P%d: ",i+1);
15         scanf("%d",&bt[i]);
16     }
17
18     for(i=0;i<n;i++)
19     {
20         for(j=i+1;j<n;j++)
21         {
22             if(bt[i]>bt[j])
23             {
24                 temp=bt[i];
25                 bt[i]=bt[j];
26                 bt[j]=temp;
27
28                 temp=p[i];
29                 p[i]=p[j];
30                 p[j]=temp;
31             }
23-
24
25
26
27
28
29
30
31
```

Enter number of processes: 4

Burst time P1: 6 3

Burst time P2: Burst time P3: 7 9 8 16

Burst time P4: 3 0

Process	BT	WT	TAT
---------	----	----	-----

P2	3	0	3
----	---	---	---

P1	6	3	9
----	---	---	---

P3	7	9	16
----	---	---	----

P4	9	16	25
----	---	----	----

=== Code Execution Successful ===

```
33
34         temp = p[i];
35         p[i] = p[j];
36         p[j] = temp;
37     }
38 }
39 }
40
41 wt[0] = 0;
42 for(i=1; i<n; i++)
43     wt[i] = wt[i-1] + bt[i-1];
44
45 for(i=0; i<n; i++)
46     tat[i] = wt[i] + bt[i];
47
48 printf("\nProcess\tBT\tPriority\tWT\tTAT\n");
49
50 for(i=0; i<n; i++)
51 {
52     printf("P%d\t%d\t%d\t\t%d\t%d\n",
53         p[i], bt[i], pr[i], wt[i], tat[i]);
54 }
55
56 return 0;
57 }
```

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2
3 int main()
4 {
5     FILE *f1, *f2;
6     char ch;
7
8     f1 = fopen("input.txt", "r");
9     f2 = fopen("output.txt", "w");
10
11     while((ch = fgetc(f1)) != EOF)
12         fputc(ch, f2);
13
14     fclose(f1);
15     fclose(f2);
16
17     printf("File Copied Successfully");
18     return 0;
19 }
```

Segmentation fault

=== Code Exited With Errors ===



```
1  #include <stdio.h>
2  #include <pthread.h>
3
4  void* threadFunction(void* arg) {
5      char* message = (char*)arg;
6      printf("%s\n", message);
7      return NULL;
8  }
9
10 int main() {
11     pthread_t thread1, thread2;
12
13     char* message1 = "Hello from Thread 1!";
14     char* message2 = "Hello from Thread 2!";
15
16     // Create threads
17     pthread_create(&thread1, NULL, threadFunction, message1);
18     pthread_create(&thread2, NULL, threadFunction, message2);
19
20     // Wait for threads to complete
21     pthread_join(thread1, NULL);
22     pthread_join(thread2, NULL);
23
24     return 0;
25 }
```

Hello from Thread 1!

Hello from Thread 2!

=== Code Execution Successful ===

```
33         time+=rt[i];
34         rt[i]-=tq;
35     }
36     else
37     {
38         time+=rt[i];
39         wt[i]=time-bt[i];
40         rt[i]=0;
41     }
42 }
43 }
44
45 }while(!done);
46
47 for(i=0;i<n;i++)
48     tat[i]=wt[i]+bt[i];
49
50 printf("\nProcess\tBT\tWT\tTAT\n");
51
52 for(i=0;i<n;i++)
53     printf("P%d\t%d\t%d\t%d\n",
54         i+1,bt[i],wt[i],tat[i]);
55
56 return 0;
57 }
```



```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i, j;
6     int bt[10], pr[10], wt[10], tat[10], p[10];
7     int temp;
8
9     printf("Enter number of processes: ");
10    scanf("%d", &n);
11
12    for(i=0; i<n; i++)
13    {
14        p[i] = i + 1;
15        printf("Enter Burst Time and Priority of P%d: ", i+1);
16        scanf("%d%d", &bt[i], &pr[i]);
17    }
18
19    // Sort by Priority (Smaller number = Higher priority)
20    for(i=0; i<n-1; i++)
21    {
22        for(j=i+1; j<n; j++)
23        {
24            if(pr[i] > pr[j])
25            {
26                temp = pr[i];
27                pr[i] = pr[j];
28                pr[j] = temp;
29
30                temp = bt[i];
31                bt[i] = bt[j];
32                bt[j] = temp;
```

```
Enter number of processes: 4
Enter Burst Time and Priority of P1: 5 2
Enter Burst Time and Priority of P2: 3 1
Enter Burst Time and Priority of P3: 8 4
Enter Burst Time and Priority of P4: 6 3
```

Process	BT	Priority	WT	TAT
P2	3	1	0	3
P1	5	2	3	8
P4	6	3	8	14
P3	8	4	14	22

=== Code Execution Successful ===

```
29     }
30 }
31
32 int main() {
33     int block[20], process[20], n, m, choice;
34
35     printf("Number of memory partitions: ");
36     scanf("%d", &n);
37
38     printf("Number of processes: ");
39     scanf("%d", &m);
40
41     printf("Enter memory partitions:\n");
42     for (int i = 0; i < n; i++)
43         scanf("%d", &block[i]);
44
45     printf("Enter process sizes:\n");
46     for (int i = 0; i < m; i++)
47         scanf("%d", &process[i]);
48
49     printf("\n1. First Fit\n2. Best Fit\n3. Worst Fit\n");
50     printf("Enter your choice: ");
51     scanf("%d", &choice);
52
53     if (choice >= 1 && choice <= 3)
54         allocate(block, process, n, m, choice);
55     else
56         printf("Invalid choice");
```



```
33     time++;
34
35     if(rt[highest]==0)
36     {
37         completed++;
38         tat[highest]=time;
39         wt[highest]=tat[highest]-bt[highest];
40     }
41 }
42
43 printf("\nProcess\tBT\tPriority\tWT\tTAT\n");
44
45 for(i=0;i<n;i++)
46     printf("P%d\t%d\t%d\t\t%d\t%d\n",
47         i+1,bt[i],pr[i],wt[i],tat[i]);
48
49 return 0;
```

```
    );  
}  
  
// Wait for threads to finish  
for (int i = 0; i < NUM_PHILOSOPHERS; i++) {  
    pthread_join(philosophers[i], NULL);  
}  
  
// Destroy mutex locks  
for (int i = 0; i < NUM_PHILOSOPHERS; i++) {  
    pthread_mutex_destroy(&chopsticks[i]);  
}  
  
return 0;
```

```
Philosopher 1 is thinking...  
Philosopher 0 is eating...  
Philosopher 3 is thinking...  
Philosopher 4 is eating...  
Philosopher 2 is eating...  
Philosopher 0 is thinking...  
Philosopher 2 is thinking...  
Philosopher 1 is eating...  
Philosopher 4 is thinking...  
Philosopher 3 is eating...  
Philosopher 1 is thinking...  
Philosopher 0 is eating...  
Philosopher 3 is thinking...  
Philosopher 2 is eating...  
Philosopher 0 is thinking...
```

main.c



Share

Run

Output

Clear

```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main()
5  {
6      fork();
7
8      printf("Process ID = %d\n", getpid());
9      printf("Parent ID  = %d\n", getppid());
10
11     return 0;
12 }
```

Process ID = 8402

Parent ID = 8395

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
29
30     printf("File copied successfully.\n");
31 }
32
33 int main() {
34     int choice;
35
36     printf("1. Create\n2. Copy\n3. Delete\n");
37     printf("Enter your choice: ");
38     scanf("%d", &choice);
39
40     switch (choice) {
41         case 1:
42             create();
43             break;
44
45         case 2:
46             copy();
47             break;
48
49         case 3:
50             remove("sasi.txt");
51             printf("File deleted successfully.\n");
52             break;
53
54         default:
55             printf("Invalid choice.\n");
56     }
57 }
```

```
* 1. Create
  2. Copy
  3. Delete
  Enter your choice: 1
```

```
=== Code Execution Successful ===
```



```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i, j, temp;
6     int bt[10], p[10];
7
8     printf("Enter number of processes: ");
9     scanf("%d", &n);
10
11     for(i=0; i<n; i++)
12     {
13         p[i] = i + 1;
14         printf("Enter Burst Time of P%d: ", i+1);
15         scanf("%d", &bt[i]);
16     }
17
18     // Sort according to burst time
19     for(i=0; i<n-1; i++)
20     {
21         for(j=i+1; j<n; j++)
22         {
23             if(bt[i] > bt[j])
24             {
25                 temp = bt[i];
26                 bt[i] = bt[j];
27                 bt[j] = temp;
28
29                 temp = p[i];
30                 p[i] = p[j];
31                 p[j] = temp;
32             }
23 }
```

```
Enter number of processes: 3
Enter Burst Time of P1: 45
Enter Burst Time of P2: 32
Enter Burst Time of P3: 18
```

```
Execution Order:
P3 P2 P1
```

```
=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4 #include <unistd.h>
5
6 #define SIZE 5
7 #define ITEMS 10
8
9 int buffer[SIZE];
10 int in = 0, out = 0;
11
12 sem_t empty, full;
13 pthread_mutex_t mutex;
14
15 void *producer(void *arg) {
16     for (int i = 1; i <= ITEMS; i++) {
17         sem_wait(&empty);
18         pthread_mutex_lock(&mutex);
19
20         buffer[in] = i;
21         printf("Produced: %d\n", i);
22         in = (in + 1) % SIZE;
23
24         pthread_mutex_unlock(&mutex);
25         sem_post(&full);
26
27         sleep(1);
28     }
```

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Produced: 4
Consumed: 3
Produced: 5
Produced: 6
Consumed: 4
Produced: 7
Produced: 8
Consumed: 5
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
```

```
=== Code Execution Successful ===
```

main.py



Run

Output

Clear

```
1 text = input("Enter text: ").upper()
2 key = input("Enter key: ").upper()
3
4 result = ""
5
6 for i in range(len(text)):
7     if text[i].isalpha():
8         shift = ord(key[i % len(key)]) - 65
9         result += chr((ord(text[i]) - 65 + shift) % 26 + 65)
10    else:
11        result += text[i]
12
13 print("Encrypted Text:", result)
```

```
Enter text: HELLO
Enter key: KEY
Encrypted Text: RIJVS
```

```
=== Code Execution Successful ===
```

main.py



Run

Output

Clear

```
1 key = input("Enter keyword: ").upper()
2
3 letters = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
4 matrix = ""
5
6 for ch in key:
7     if ch not in matrix and ch != "J":
8         matrix += ch
9
10 for ch in letters:
11     if ch not in matrix:
12         matrix += ch
13
14 print("Playfair Matrix:")
15
16 for i in range(0, 25, 5):
17     print(matrix[i:i+5])
```

```
Enter keyword: MONARCHY
Playfair Matrix:
MONAR
CHYBD
EFGIK
LPQST
UVWXZ
```

```
=== Code Execution Successful ===
```



```

57     return 1;
58 }
59
60 int main() {
61     int process, request[R];
62
63     printf("Enter process number (0 to 4): ");
64     scanf("%d", &process);
65
66     printf("Enter resource request: ");
67     for (int i = 0; i < R; i++)
68         scanf("%d", &request[i]);
69
70     for (int i = 0; i < R; i++) {
71         if (request[i] > available[i] ||
72             request[i] > maximum[process][i] - allocation[process][i]) {
73             printf("Request denied.\n");
74             return 0;
75         }
76     }
77
78     for (int i = 0; i < R; i++) {
79         available[i] -= request[i];
80         allocation[process][i] += request[i];
81     }
82
83     if (safe())
84         printf("Request granted.\n");

```

```

* Enter process number (0 to 4): 2
Enter resource request: 1
0
1
Request denied.

```

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1  #include <stdio.h>
2
3  int main() {
4      char path[] = "C:/Users/itssk/OneDrive/Desktop/os/example.txt";
5
6      FILE *file = fopen(path, "w");
7
8      if (file == NULL) {
9          printf("Error creating file.\n");
10         return 1;
11     }
12
13     fprintf(file, "This is an example file content.");
14     fclose(file);
15
16     printf("File created successfully: %s\n", path);
17
18     return 0;
19 }
```

Error creating file.

=== Code Exited With Errors ===

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n,i,j,time=0,completed=0;
6     int bt[10],rt[10],pr[10],wt[10]={0},tat[10];
7     int highest;
8
9     printf("Enter number of processes: ");
10    scanf("%d",&n);
11
12    for(i=0;i<n;i++)
13    {
14        printf("Enter BT and Priority P%d: ",i+1);
15        scanf("%d%d",&bt[i],&pr[i]);
16        rt[i]=bt[i];
17    }
18
19    while(completed<n)
20    {
21        highest=-1;
22
23        for(i=0;i<n;i++)
24        {
25            if(rt[i]>0)
26            {
27                if(highest==-1 || pr[i]<pr[highest])
28                    highest=i;
29            }
30        }
31
32        rt[highest]--;
```

```
Enter number of processes: 4
Enter BT and Priority P1: 5
2
```

```
Enter BT and Priority P2: 3 1
Enter BT and Priority P3: 8 4
Enter BT and Priority P4: 6 3
```

Process	BT	Priority	WT	TAT
P1	5	2	3	8
P2	3	1	0	3
P3	8	4	14	22
P4	6	3	8	14

```
=== Code Execution Successful ===
```

main.py



Share

Run

Output

Clear

```
1 plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
2 cipher = "QWERTYUIOPASDFGHJKLZXCVBNM"
3
4 text = input("Enter text: ").upper()
5
6 result = ""
7
8 for ch in text:
9     if ch in plain:
10         result += cipher[plain.index(ch)]
11     else:
12         result += ch
13
14 print("Encrypted Text:", result)
```

Enter text: HELLO
Encrypted Text: ITSSG

=== Code Execution Successful ===



```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n,i;
6     int bt[10],wt[10],tat[10];
7
8     printf("Enter number of processes: ");
9     scanf("%d",&n);
10
11     for(i=0;i<n;i++)
12     {
13         printf("Burst Time P%d: ",i+1);
14         scanf("%d",&bt[i]);
15     }
16
17     wt[0]=0;
18
19     for(i=1;i<n;i++)
20         wt[i]=wt[i-1]+bt[i-1];
21
22     for(i=0;i<n;i++)
23         tat[i]=wt[i]+bt[i];
24
25     printf("\nP\tBT\tWT\tTAT\n");
26
27     for(i=0;i<n;i++)
28         printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
29
30     return 0;
31 }
```

Enter number of processes: 4
Burst Time P1: 12
Burst Time P2: 14
Burst Time P3: 15
Burst Time P4: 16

P	BT	WT	TAT
P1	12	0	12
P2	14	12	26
P3	15	26	41
P4	16	41	57

=== Code Execution Successful ===

```
32     }
33 }
34 }
35
36 printf("\nExecution Order:\n");
37
38 for(i=0; i<n; i++)
39 {
40     printf("P%d ", p[i]);
41 }
42
43 return 0;
44 }
```

main.py



Share

Run

Output

Clear

```
1 text = input("Enter text: ").upper()
2 a = int(input("Enter a: "))
3 b = int(input("Enter b: "))
4
5 result = ""
6
7 for ch in text:
8     if ch.isalpha():
9         p = ord(ch) - 65
10        c = (a * p + b) % 26
11        result += chr(c + 65)
12    else:
13        result += ch
14
15 print("Encrypted Text:", result)
```

Enter text: HELLO

Enter a: 5

Enter b: 8

Encrypted Text: RCLLA

=== Code Execution Successful ===

```
32     }
33 }
34
35 wt[0]=0;
36
37 for(i=1;i<n;i++)
38     wt[i]=wt[i-1]+bt[i-1];
39
40 for(i=0;i<n;i++)
41     tat[i]=wt[i]+bt[i];
42
43 printf("\nProcess\tBT\tWT\tTAT\n");
44
45 for(i=0;i<n;i++)
46     printf("P%d\t%d\t%d\t%d\n",
47         p[i],bt[i],wt[i],tat[i]);
48
49 return 0;
50 }
```




```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n,i,tq,time=0,done;
6     int bt[10],rt[10],wt[10]={0},tat[10];
7
8     printf("Enter number of processes: ");
9     scanf("%d",&n);
10
11     for(i=0;i<n;i++)
12     {
13         printf("Burst time P%d: ",i+1);
14         scanf("%d",&bt[i]);
15         rt[i]=bt[i];
16     }
17
18     printf("Enter Time Quantum: ");
19     scanf("%d",&tq);
20
21     do
22     {
23         done=1;
24
25         for(i=0;i<n;i++)
26         {
27             if(rt[i]>0)
28             {
29                 done=0;
30
31                 if(rt[i]>tq)
32                 {
```

```
Enter number of processes: 4
Burst time P1: 5
Burst time P2: 4
Burst time P3: 2
Burst time P4: 1
Enter Time Quantum: 2
```

Process	BT	WT	TAT
P1	5	7	12
P2	4	7	11
P3	2	4	6
P4	1	6	7

=== Code Execution Successful ===

```

29     return NULL;
30 }
31
32 void *consumer(void *arg) {
33     for (int i = 1; i <= ITEMS; i++) {
34         sem_wait(&full);
35         pthread_mutex_lock(&mutex);
36
37         printf("Consumed: %d\n", buffer[out]);
38         out = (out + 1) % SIZE;
39
40         pthread_mutex_unlock(&mutex);
41         sem_post(&empty);
42
43         sleep(2);
44     }
45     return NULL;
46 }
47
48 int main() {
49     pthread_t producer_thread, consumer_thread;
50
51     sem_init(&empty, 0, SIZE);
52     sem_init(&full, 0, 0);
53     pthread_mutex_init(&mutex, NULL);
54
55     pthread_create(&producer_thread, NULL, producer, NULL);
56     pthread_create(&consumer_thread, NULL, consumer, NULL);

```

```

* Produced: 1
  Consumed: 1
  Produced: 2
  Consumed: 2
  Produced: 3
  Produced: 4
  Consumed: 3
  Produced: 5
  Produced: 6
  Consumed: 4
  Produced: 7
  Produced: 8
  Consumed: 5
  Produced: 9
  Produced: 10
  Consumed: 6
  Consumed: 7
  Consumed: 8
  Consumed: 9
  Consumed: 10

```

=== Code Execution Successful ===